

What's Agile

- A family of software development methodologies that produce software in *short iterations* and allow *greater change in design*.

Agile Methodologies

Core Values

- **Individuals and interactions > processes and tools**
 - Communication, creativity, collaboration, and quick feedback matter more than strict procedures and workflow.
- **Working software > comprehensive documentation**
 - A functioning product is proof of progress and value; documentation is great but quickly becomes outdated.
- **Customer collaboration > contract negotiation**
 - By direct collaboration, we make sure that the product always follows the changing needs of customers.
- **Responding to change > following a plan**
 - As the world, technologies, market, and user needs change, so does the plan need to change (flexibility over rigidly following a plan).

Agile Principles

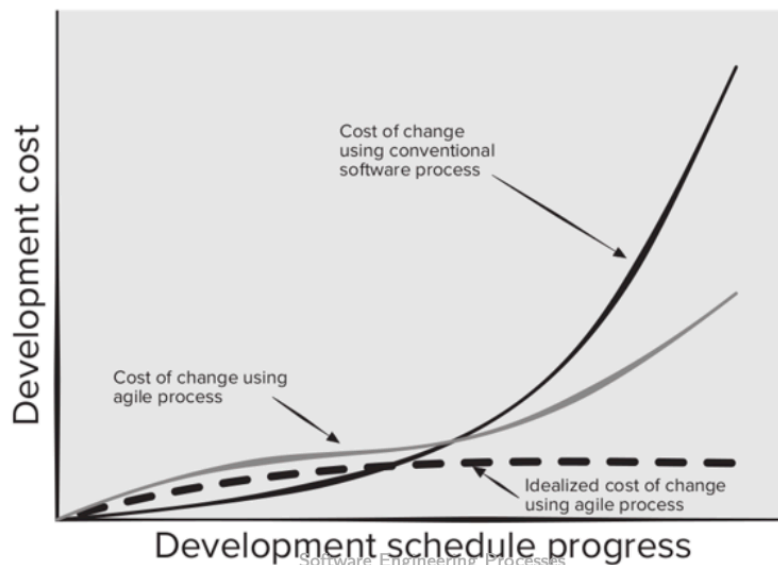
- **Priority:** Satisfy the customer with early and continuous delivery of a valuable product.
- **Changing Requirements:** Welcome changing requirements even late in development.
- **Frequency:** Deliver working software frequently.
- **Collaboration:** Business people and developers must work together daily.
- **Individuals + Environment:** Build projects around trusted, motivated individuals and give them the right environment.
- **Face to Face:** The best way to get information is face-to-face conversation with the development team.
- **Measure of Progress:** Working software.
- **Sustainable Development:** Sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- **Attention to Excellence:** Continuous attention to technical excellence and good design.

- **Simplicity:** Focus on high-value tasks and delay less important ones that could complicate the project.
- **Self-Organizing Teams:** Give the best architectures, requirements, and designs.
- **Reflection:** At regular intervals, teams reflect on how to enhance effectiveness and adjust their behavior accordingly.

Agile Principles Rephrased (3lash kayn bzzf)

- **Short Releases and Iterations:** Each time, release a sprint with new functionalities or fixed bugs.
- **Incremental Design:** Improve the existing design when more knowledge is acquired.
- **User Involvement:** Keep the customers closely involved throughout the development process by asking for constant feedback.
- **Embrace Change:** Requirements may change → design and system will accommodate changes.
- **People, Not Process:** Recognize team skills and let them develop their own ways of working.
- **Maintain Simplicity:** Simplicity in both development and communication, and work to eliminate complexity from the system.

Agile Processes vs Traditional Processes when it comes to Handling Changes



• bermacine

October 15, 2025

Rational Unified Process

- Is a software process framework developed by Rational Software Corporation, later acquired by IBM.
- Driven by: use cases/requirements, architecture-centric approach, and iterative increments.

- Goes through four phases:
 - **Inception**
 - Planning, clarifying the goals and scope.
 - The scenarios/use cases that drive the architecture and design.
 - Schedule, resources.
 - Planning for implementation, testing, integration, and configuration.
 - Estimate potential risks.
 - **Elaboration** (most critical phase)
 - Establish the critical requirements/baseline design.
 - Establish platforms and methodologies for implementation, testing, and integration.
 - Establish major test scenarios and metrics for the agreed goals.
 - **Construction Phase**
 - Complete most of the implementation on time; the code should be releasable and restricted to a set of users.
 - Establish remaining activities that have to be completed to achieve the goal.
 - **Transition Phase**
 - All fixes and components are integrated.
 - Non-code artifacts are integrated into a complete product.
 - This phase establishes a final, ready-to-use, ready-to-support software release, including deployment, migration, and possible changes to business procedure strategy.

Scrum

****Definition**

- a project management framework that is built upon the intelligence of people using, their relationships and interactions, not detailed instructions
- emphasises teamwork, collaboration, communication and transparency, and continual improvement in self organized teams
- it is iterative and incremental

****Why**

- **Select and prioritize** it selects and prioritizes the right functionalities in the right increment

- **Environment** created with transparency, communication, collective responsibility, continuous progress
- **Deliverables** project is broken down to manageable, understandable deliverables
- **Unstable Requirements** don't hold up the progress, focus on new features
- **On time Delivery** for customers to gain feedback
- **Visibility** the whole team has visibility of everything, improves team communication and morals

Scrum Members

- **Committed**
 - Members that are fully committed; teams of fewer than 10 for better communication and productivity.
 - Are accountable for creating a valuable increment at every sprint.
- **Involved**
 - Members who have an interest in the success of the project: executives, customers, etc.

Scrum Roles

- **Product Owner** (one person)
 - Sets the vision of the product and its *primary goal*.
 - Communicates with *clients, end users, and stakeholders* to understand needs and get feedback.
 - Has insight on how to *increase the value* of the product.
 - Is accountable for creating the product backlog:
 - *Creates, orders, and communicates* backlog items.
 - Ensures the product backlog is *transparent, visible, and understood*.
 - *Approves the sprint product* by validating that it meets the intended value.
- **Scrum Master (Servant Leadership)**
 - Ensures *smooth communication* between the product owner and the development team.
 - Ensures the team *progresses on time*, meeting deadlines, delivering, and removing obstacles.
 - Ensures the team follows the Scrum methodology — *trains, teaches, leads daily meetings, and provides feedback*.
- **Development Team**
 - Executes the sprint under the *vision* of the product owner and the *direction* of the Scrum Master.

- Cross-functional members with the responsibility to turn the *backlog items they select* into a valuable increment.
- Responsible for *managing the progress* of work during a sprint.

Scrum Artifacts

- **Product Backlog**
 - List of features, functions, requirements, enhancements, and fixes to be made in *future releases*.
 - Has descriptions, order, estimated duration, value, and test descriptions to prove completeness when "done."
 - *Backlog item refinement* is the “divide and conquer” process for a backlog item.
 - Maintained and managed by the *Product Owner*.
 - Items are in the form of:
 - *User Story*: a feature from the end-user perspective.
 - *Action Task*: an action item to be executed or implemented.
 - *Epic*: a set of related stories that correspond to a *major feature or component*.
- **Sprint Backlog**
 - Sprint goal + tasks that must be done by the sprint deadline.
 - *Timebox*: the time frame for the task/sprint (for example, 2 → 4 weeks).
 - Tasks are *taken from the product backlog*, prioritized, and assigned to members of the development team during meetings.
 - Depending on the state of the project, tasks may be *added, removed, or re-prioritized* in the sprint backlog.
- **Product Increment**
 - The set of all tasks, user stories, and product backlog items that are *developed* during sprints, marked as complete (based on the definition of “done”), and made available to the end user.
 - Product increments are built upon each other until the team completes the project.

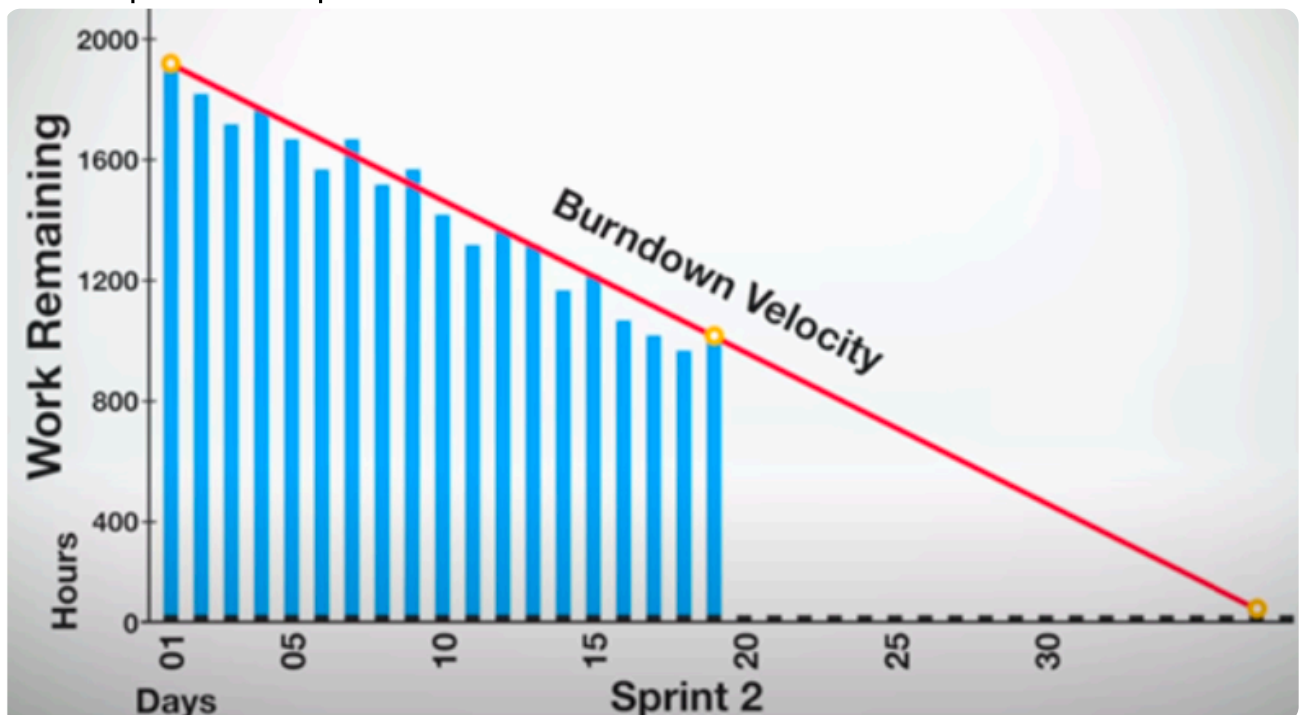
Scrum Ceremonies

- Meetings/events to *inspect and adapt* Scrum artifacts to improve transparency.
- Scrum events are held *at the same time, same place*.
- Examples: sprint planning, daily Scrum, sprint review, sprint retrospective (how the last sprint went, how to improve).
- **Sprint Planning**

- Start of sprint / goal: select items from the product backlog to implement.
- All members attend, chaired by the *Product Owner*.
- Three most important points: why is the sprint *valuable*, what can be *done*, how will it *be done*.
- **Daily Scrum**
 - Daily brief (15 mins) for developers only, to discuss:
 - What did I do *yesterday/today*?
 - Are there any *impediments* in my way?
- **Sprint Review**
 - To *inspect* the outcome of the sprint and determine *future adaptations*.
 - Scrum team presents the work to *key stakeholders*; progress toward the product goal is discussed.
 - Attendees *collaborate* on what *to do next*; product backlog may be adjusted.
 - Max 4 hours for a 1-month sprint.
- **Sprint Retrospective**
 - To plan ways to *increase quality and effectiveness*.
 - Held by the *Scrum team*; reviews how the sprint went regarding individuals, progress, tools, and definition of done.
 - Max 3 hours for a 1-month sprint.

Scrum Burndown Chart

- Used to predict if a sprint can be finished on time.



Five Values of Scrum

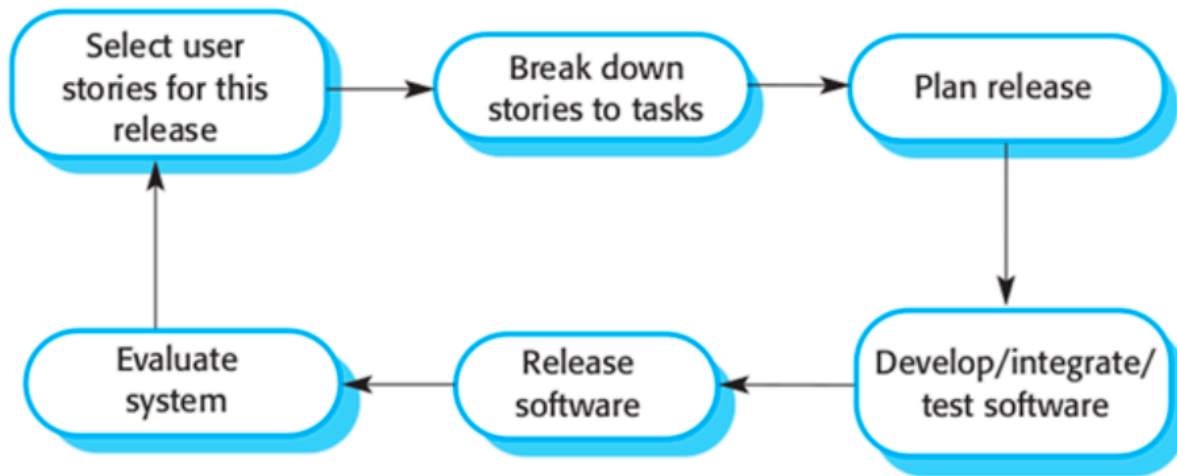
- **Commitment**
- **Focus**
- **Openness**
- **Respect**
- **Courage**

Steps on How to Get Started Using Scrum

- **Setup** tools/platforms: Jira, Slack, Google Calendar.
- **Assign** Product Owner and Scrum Master.
- **Requirements** gathered by the PO and turned into tasks/stories in the product backlog.
- **Timing Ceremonies** must be fixed by the SM.
- **Iterations**
 - *Sprint Planning Meeting*
 - Discuss the items to execute.
 - Assign them to developers.
 - *Sprint Execution*
 - Daily Scrum meeting.
 - Development.
 - Integration.
 - Testing.
 - Acceptance based on DoD (Definition of Done).

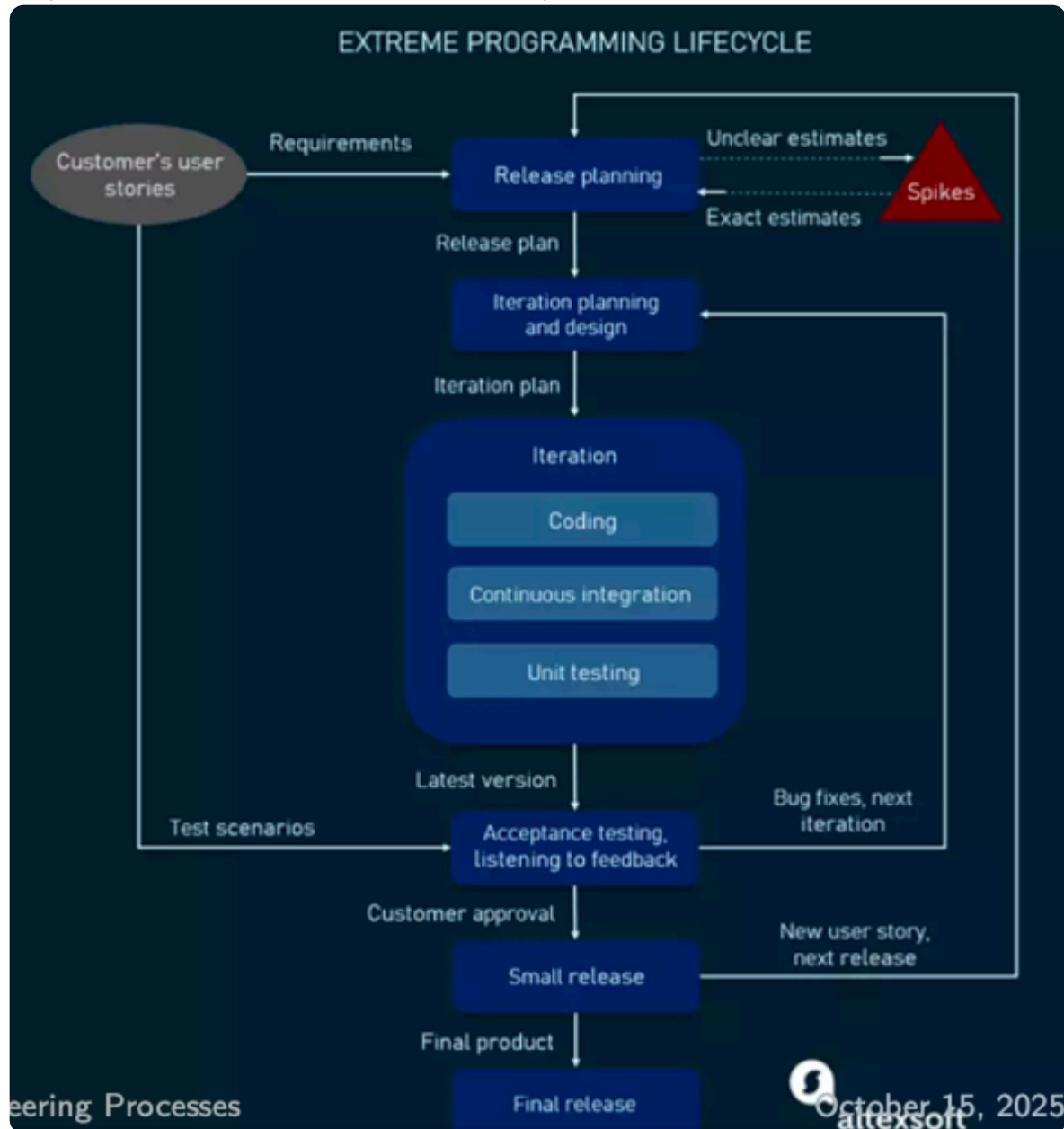
Extreme Programming (XP)

XP Process Model



- It's the most used Agile process.
- It uses *user stories* as requirements, *test-driven* development, and *pair programming*.
- *Pair programming*: two programmers work together on one computer — one is the *driver* (types the code), and the other is the *navigator* (reviews, suggests, and

inspects). It acts as an informal review process.



XP Principles

- **Code Standard:** consistent code style ensures consistency, maintainability, and readability.
- **Refactoring:** changing the structure of code without changing its behavior; improves readability and maintainability, removes redundancy, and eliminates unnecessary business logic.
- **Pair Programming:** the first (*driver*) codes, the second (*navigator*) reviews, inspects, suggests improvements, and fixes mistakes.
- **Test-Driven Development (TDD):** automated tests are written *before the code itself*.
- **Collective Code Ownership:** every member can review and update code.

- **Continuous Integration (CI)**: after every code compilation and build, integrate it; then all tests are executed automatically for the entire system.
- **Small Release**: allows frequent feedback.
- **Onsite Customer**: helps answer questions, craft vision, set priorities and plan releases, define user stories and acceptance criteria, and resolve disputes.
- **Sustainable Pace**: no overtime.
- **System Metaphor**: a story that everyone can tell about how the system works.
- **Simple Design**: start with a simple design, then develop with iterations.
- **Planning Game**: at the beginning of each iteration cycle, the dev team and customers discuss and approve *product features*, then developers plan the upcoming iteration and release, and assign tasks.

Benefits and Drawbacks

Benefits

- Stable system.
- Clear and clean code.
- *Fast delivery* of features.
- No overtime.
- Better collaboration (pair programming).
- Good transparency and visibility (collective code ownership).
- Customer satisfaction (due to customer testing).

Drawbacks

- Unclear estimates for time, budget, etc.
- Time waste.
- Lack of documentation.
- Pair programming takes longer.
- Lack of long-term vision.

Agile Methodologies Challenges

- Problems with the *informality* of Agile vs. the *legal approach* of contract definition.
- Agile is:
 - Appropriate for *new software development* rather than *software maintenance*.
 - In large companies, the majority of software costs come from *maintaining existing software*.

Conclusion

- There is *no best* software process.
- No software process *works for all* projects.
- Software processes must be *selected, adapted, readjusted, and applied* as appropriate for each *project, team, and organizational context*.